

Desenvolvendo o "Game Vault" com Express, EJS, Bootstrap e SQLite

Este guia detalha os passos para a construção do **Game Vault** (Cofre de Jogos), um catálogo pessoal onde os usuários poderão cadastrar os jogos que possuem, atribuir notas e definir seus status.

1. Preparação do Ambiente e Inicialização do Banco de Dados

Comece inicializando o seu projeto e instalando as bibliotecas essenciais. No terminal, dentro da pasta do projeto, execute:

```
npm init -y
npm install express ejs sqlite3
```

Abaixo está o esqueleto do seu arquivo principal, index.js. Ele já possui a configuração básica do servidor web e a estrutura para conectar ao banco de dados. Sua tarefa é escrever o comando SQL que constrói a tabela na inicialização.

```
const express = require('express');
const sqlite3 = require('sqlite3').verbose();
//const sqlite3 = require('node:sqlite');

const app = express();
const PORTA = 3000;

// Configuração do motor de visualização e recebimento de dados
app.set('view engine', 'ejs');
app.use(express.urlencoded({ extended: true }));

// Inicialização e conexão com o banco de dados
const db = new sqlite3.Database('gamevault.db', (err) => {
  if (err) {
    console.error('Erro ao conectar com o banco de dados:', err.message);
  } else {
    console.log('Conectado ao banco de dados SQLite.');

// TODO: Escreva a query SQL para criar a tabela 'jogos'



// Dica: Use o comando CREATE TABLE IF NOT EXISTS.



// A tabela deve ter as colunas:



// - id (INTEGER PRIMARY KEY AUTOINCREMENT)



// - titulo (TEXT)



// - plataforma (TEXT)


```

```

    // - status (TEXT)
    // - nota (INTEGER)
    // - capa_url (TEXT)

    const queryCriacao = ``; // Escreva sua query aqui

    db.run(queryCriacao, (err) => {
      if(err) console.error("Erro ao criar tabela:", err.message);
    });
  }
});

// O servidor ficará ouvindo a porta 3000
app.listen(PORTA, () => {
  console.log(`Servidor rodando em http://localhost:${PORTA}`);
});

```

2. Estruturando a Interface Visual com Bootstrap

Para não repetirmos o cabeçalho e o rodapé em todas as páginas, usaremos *partials* do EJS.

Crie uma pasta chamada `views` e, dentro dela, uma subpasta `partials`.

Você precisará criar dois arquivos: `header.ejs` e `footer.ejs`.

O que fazer no `header.ejs`:

- Crie a estrutura básica do HTML (`<!DOCTYPE html>`, `<head>`, `<body>`).
- No `<head>`, adicione o link do CDN do Bootstrap (busque na documentação oficial do Bootstrap 5).
- No `<body>`, crie uma barra de navegação (Navbar) contendo links para a Página Inicial (`/`) e para Adicionar Jogo (`/cadastrar`).
- Abra uma `<div class="container">` que vai abraçar o conteúdo de todas as páginas.

O que fazer no `footer.ejs`:

- Feche a `</div>` do container que você abriu no header.
- Adicione um rodapé simples com seus créditos.
- Feche as tags `</body>` e `</html>`.

3. Criando o Formulário de Cadastro e Lidando com Imagens

Agora, precisamos criar as rotas no `index.js` e o arquivo visual para receber os dados do usuário.

Adicione os esqueletos das rotas abaixo no seu `index.js`:

```

// Rota GET para exibir o formulário
app.get('/cadastrar', (req, res) => {
  // TODO: Use res.render() para renderizar a página 'cadastro.ejs'

```

```
});

// Rota POST para processar o formulário
app.post('/adicionar', (req, res) => {
  // Os dados do formulário chegam aqui através do req.body
  const { titulo, plataforma, status, nota, capa_url } = req.body;

  // TODO: Escreva a query SQL de inserção
  // Dica: Use INSERT INTO... e não se esqueça dos placeholders (?)
  const query = ``;

  // TODO: Execute a query usando db.run()
  // Passe o array com as variáveis na ordem correta: [titulo, plataforma,
status, nota, capa_url]
  // No callback de sucesso, use res.redirect('/') para voltar à página
principal.
});
```

O que fazer no views/cadastro.ejs:

- Inclua o header no topo e o footer no final usando <%- include('partials/header') %>.
- Crie um <form> configurado com action="/adicionar" e method="POST".
- Crie os campos (<input> e <select>) com as classes do Bootstrap (form-control, form-select).
- **Atenção:** Os atributos name dos inputs devem ter os nomes exatos esperados pelo backend (titulo, plataforma, status, nota, capa_url).

Dica valiosa sobre imagens: Você não precisa implementar upload de arquivos. Peça para o usuário colar o link de uma imagem da internet no campo capa_url. O HTML consegue carregar imagens remotas perfeitamente.

4. O Catálogo Visual e o Sistema de Grid

Esta é a página principal da aplicação. Ela deve buscar os dados no banco e gerar uma interface de catálogo (vitrine) iterando sobre os jogos.

No seu index.js, configure a rota principal:

```
app.get('/', (req, res) => {

  // TODO: Escreva a query SQL para buscar todos os jogos
  const query = ``;

  // TODO: Execute a query usando db.all()
  // No callback, caso não haja erro, use res.render() para renderizar a página
  'index.ejs'
  // Lembre-se de passar o array de resultados (rows) em um objeto, por exemplo:
```

```
{ jogos: rows }  
  
});
```

O que fazer no views/index.ejs:

Nesta página, você vai iterar sobre a variável jogos enviada pelo backend. Você deve construir um layout de Grid do Bootstrap para exibir *Cards*.

Aqui está um esqueleto estrutural importante para evitar que o layout quebre. Preste atenção onde o laço de repetição (forEach) deve entrar:

```
<%- include('partials/header') %>  
  
<h1>Meu Cofre de Jogos</h1>  
  
<div class="row">  
  <div class="col-md-4 mb-4">  
    <div class="card h-100">  
      <div class="card-body">  
      </div>  
    </div>  
  </div>  
</div>  
  
<%- include('partials/footer') %>
```

5. Implementando a Exclusão de Registros

Na interface anterior, você criou um link (botão) que aponta para a rota de exclusão com o ID do jogo na URL. Agora, você deve capturar essa requisição e deletar o registro correspondente no banco.

No seu index.js:

```
// Usamos GET aqui para simplificar a exclusão direta via tag <a> do HTML.  
app.get('/deletar/:id', (req, res) => {  
  // Capturamos o parâmetro dinâmico da URL  
  const idDoJogo = req.params.id;  
  
  // TODO: Escreva a query SQL para deletar um registro específico  
  // Dica: DELETE FROM ... WHERE ...  
  const query = ``;  
  
  // TODO: Execute a query usando db.run() passando o [idDoJogo]  
  // No callback, após deletar com sucesso, use res.redirect('/')  
});
```

